StateScout: Auditing What a User Interface Must Not Contain, and What It Must

Kavinraj S[1], Harshita R[2], Sanjay A R[3], Jai Subiksha T[4], and Sathiya R R[5]

[1] Department of Computer Science and Engineering, Amrita School of Computing Coimbatore, Amrita Vishwa Vidyapeetham, India
cb.sc.u4cse23329@cb.students.amrita.edu
[2] Department of Computer Science and Engineering, Amrita School of Computing Coimbatore, Amrita Vishwa Vidyapeetham, India
cb.sc.u4cse23028@cb.students.amrita.edu
[3] Department of Computer Science and Engineering, Amrita School of Computing Coimbatore, Amrita Vishwa Vidyapeetham, India
cb.sc.u4cse23052@cb.students.amrita.edu
[4] Department of Computer Science and Engineering, Amrita School of Computing Coimbatore, Amrita Vishwa Vidyapeetham, India
cb.sc.u4cse23327@cb.students.amrita.edu
[5] Department of Computer Science and Engineering, Amrita School of Computing Coimbatore, Amrita Vishwa Vidyapeetham, India
rr_sathiya@cb.amrita.edu

Abstract. Web applications enforce authorization on the server, yet their interfaces can still show what a role should not see or omit what a policy requires. We present StateScout, a prototype that explores an application as one fixed role and checks each reached state against a role-scoped policy of forbidden and required elements, computed as an intersection and a set difference over a semantic state description. The evaluation is fixture-level: one six-page application with four planted violations and seven small same-project apps, using fakes. Under oracle perception the production audit agrees with the ground truth, a consistency check. A rule-based channel fails mainly by vocabulary mismatch; an in-sample label script beats it on the first app but finds 3 of 8 forbidden violations on four generated apps; and a simulated crash shows that claim-before-execute does not by itself make resumption safe. No browser or VLM pipeline is evaluated.

Keywords: web UI testing, test oracles, negation, vision-language models, state-space exploration

1 Introduction

Broken access control heads the OWASP Top 10 [32], and research tools aimed at it mostly examine the *server*: they test whether requests that should be refused are in fact refused [38, 18, 31, 30, 21]. This paper concerns a neighbouring defect class. Consider a dashboard that renders an "Admin" link for every visitor while

the backend rejects the request behind it. A request-level scanner has nothing to report, yet the link discloses that privileged functionality exists and where it lives. A debug panel left in a production build, or a destructive control that a guest reaches in a few clicks, is the same kind of defect. The converse also occurs: a page may *lack* something the organisation requires, such as a skip-to-content link, a consent banner or a logout control.

Workflow-style UI tests typically exercise what should happen, so such problems can go unnoticed. Crawlers that infer a model of the application [28, 26, 5] reach many states but usually apply generic checks, and DOM invariants [29] can assert presence or absence but are written in code, per application, with no notion of role. Agents built on language and vision-language models [22, 46, 23, 51] explore and judge screens flexibly, but the model usually delivers the verdict, which is risky when the verdict is that something is *absent*: vision-language models (VLMs) answer negated queries poorly on natural-image benchmarks [1].

We present StateScout, a design and prototype that treats this as a state-space auditing problem. Given a seed URL, a role and a policy such as "a guest must never see the admin link; every page must offer a skip-to-content link", it explores the application as that role, retains the full state graph, and checks every reached state against the policy. A finding names the clause it violates, the set operation that raised it and the evidence behind it.

Scope of the contribution. Crawling into state graphs, checking invariants on crawled states, and using VLMs in GUI testing all have substantial prior art (Sect. 2), and we claim none of them. Accessibility checkers and web scanners already flag required and forbidden artifacts one rule at a time (Sect. 2). What we offer is a formulation and a prototype that combine these threads around one question: does the UI that a given role can reach conform to a declared policy of what must and must not exist? Concretely:

1. an oracle in which clause polarity is a typed field, so that *forbidden-present* and *required-absent* are two distinct set operations over a semantic description of each state (Sect. 3);
2. an elementary observation on error direction (Proposition 2): the two operations are monotone and antitone in the perceived set, so a perceiver that under-reports can only miss forbidden elements and raise false alarms on required ones, and an over-reporter does the reverse. We use it to organise the evaluation per polarity and to motivate a design in which DOM and accessibility-tree evidence settles which *elements* exist while a VLM only annotates them. That design is not evaluated here, and annotation errors remain possible (Sect. 4.2);
3. a cyclic exploration loop that claims each (state, action) pair before executing it and restricts path replay by action kind (Sect. 4), together with measured loop-level counters and a simulated crash test whose outcome qualifies the crash-safety claim (Sect. 5.2); and
4. a fixture-level evaluation that states, arm by arm, which components it exercises and which it replaces with fakes (Sect. 5).

StateScout does not test whether the server enforces authorization. It audits what a role is shown, which is complementary. The browser-driven pipeline and the VLM channel are not evaluated here.

Organisation. Sect. 2 positions the work, Sect. 3 gives the model, Sect. 4 the prototype, Sect. 5 the experiments and their threats to validity, and Sect. 6 and 7 discuss and conclude.

2 Background and Related Work

2.1 A Running Example

Our test application, `broken-admin`, has six pages (`index`, `login`, `register`, `dashboard`, `reports`, `admin`); a guest signs in from `index` and reaches `dashboard`, which links to `reports` and `admin`, and most pages link back. The policy has four sentences, each with one planted violation: (F1/V-01) a guest must never see the admin link, planted on `dashboard`; (F2/V-02) no page may ship a debug panel, planted on `reports`; (F3/V-03) a guest must never be able to delete a user, planted as a Delete-user button on `admin`; and (R1/V-04) every page must offer a skip-to-content link, which `admin`, the only page lacking one, omits. The forbidden defects are *additional* UI, so a workflow test has no reason to fail, and R1 is an absence, which an intersection-only oracle cannot express.

2.2 Related Work

Table 1 compares the closest systems we found along the axes of our literature search (crawling and model inference, LLM/VLM GUI agents, access-control testing, negation in VLMs, specification and accessibility oracles) by web search in September and October 2026. It is a positioning aid, not a systematic review, and absence from it is not evidence that no related system exists. Cells were read from the abstract of each cited source (and from the body for ATUSA); "?" marks what we could not establish.

No row combines exploration, a declarative role-scoped policy and both polarities as typed fields; this describes the table, not all prior work. ATUSA supplies both polarities per application through code. axe-core ships fixed rules of both polarities (`bypass` requires a mechanism, `marquee` forbids an element) and ZAP ships passive rules and per-user scanning, but neither has role-scoped, user-declared clauses.

Crawlers and GUI testers. GUI Ripping [26], Crawljax [28] and WebMate [5] recover state-flow models of running interfaces; on Android, Sapienz, Stoat, APE, DroidBot and Humanoid [25, 37, 14, 19, 20] explore by search, stochastic models, model refinement or learned input generation, and Stoat reports crash detection as its headline result. Such models are cyclic by construction, so a cyclic graph is not our novelty. Our fingerprint is an ordinary state abstraction [45], and Doupé et al. [11] infer a server-side state machine from state-changing requests, which our claim rule only echoes for the auditor's own side effects.

Table 1. Positioning against nearby work. ✓: stated in the source we read; ∼: partial, or only expressible in code; –: not stated in what we read; ?: not established (MATE: we could not obtain an abstract naming its checks). AXNav's ∼ on REQ reflects behavioural heuristics that flag expected behaviour missing, not required elements. AE: autonomous exploration; POL: declarative policy as input; NEG: must-not-exist oracle; REQ: required-absent oracle; ROLE: role- or user-scoped; MM: multimodal perception. StateScout marks mean implemented and exercised at fixture level only (AE: scripted crawler; MM: unevaluated, not wired in; POL: no English compiler).

System	AE	POL	NEG	REQ	ROLE	MM
Crawljax [28]	✓	–	–	–	–	–
ATUSA [29]	✓	∼	✓	✓	–	–
WebMate [5]	✓	–	–	–	–	–
Kea [44]	✓	∼	∼	∼	–	–
MATE [12]	✓	–	?	?	–	–
Sun et al. [38]	–	–	∼	–	✓	–
DroidAgent [46]	✓	–	–	–	–	–
VisionDroid/Trident [23]	✓	–	–	–	–	✓
AXNav [39]	–	∼	–	∼	–	∼
UIGuard [3]	–	∼	✓	–	–	✓
ST-WebAgentBench [17]	–	✓	∼	∼	–	–
axe-core [7, 8, 9]	–	∼	✓	✓	–	–
ZAP [48, 50, 49]	✓	–	✓	–	∼	–
StateScout	∼	∼	✓	✓	✓	∼

Oracles, invariants and deployed web checkers. Barr et al. [2] survey the oracle problem; Memon et al. [27] introduced GUI oracles. ATUSA [29] checks generic and application-specific invariants on every state Crawljax reaches, written as DOM conditions with XPath and JavaScript preconditions, and its case studies include invariants that require a structure to exist and invariants that flag unwanted content. In expressive power our oracle therefore offers a skilled tester nothing that hand-written ATUSA invariants could not (Sect. 5.5 compares with a hand-written script of that kind, our own re-implementation and not ATUSA). The differences are that polarity is a typed field, clauses use a vocabulary of roles, names, tags and capabilities in place of page-specific XPath, clauses carry a role scope, and each finding records the operation that produced it. Kea [44] checks user-written properties and MATE [12] applies accessibility checks during Android exploration.

Both polarities are also checked on the web. The axe-core engine [7] has a `bypass` rule that passes a page offering an internal skip link, a heading or a landmark [8], implementing WCAG success criterion 2.4.1 [41], and a `marquee` rule that forbids a deprecated element [9]. The first is a deployed required-presence oracle close to our example R1, although it also accepts a heading or landmark where R1 demands a skip link; both run on whatever page a test driver supplies. ZAP has a spider that discovers resources from seed URLs [49], a passive rule that flags known debug-error fragments in responses [48], and it can issue requests as a configured user in a context [50]. Detecting a missing skip link or a leaked debug artifact is thus not new; our contribution is treating both polarities uniformly in one role-scoped, declarative clause language over an explored state graph.

Access-control testing. Sun et al. [38] statically build per-role sitemaps, compare them to find privileged pages, and check whether forced browsing to them succeeds. Li et al. [18], Noseevich and Petukhov [31], MACE [30], BACScan [21] (a feedback-driven oracle on read and modify operations) and BACFuzz [10] target authorization flaws on the server side. StateScout instead asks whether the interface shown to a role conforms to a declared policy, including a required-absent half those tools do not model. Exposure of a link is not by itself an OWASP A01 vulnerability [32], so we position the work as policy conformance.

LLM and VLM agents, negation, and specifications. GPTDroid, DroidAgent and AutoDroid [22, 46, 42] drive mobile apps with language models; VisionDroid (renamed Trident in a later arXiv version) [23] asks a multimodal model to detect non-crash functional bugs; AXNav [39], AUITestAgent [15] and WebTestPilot [40] turn natural-language descriptions into interactions and checks; UIGuard [3] detects dark patterns, a fixed taxonomy of forbidden UI. WebArena, Mind2Web, OSWorld and SeeAct [52, 6, 43, 51] concern web and computer-use agents, and ST-WebAgentBench [17] measures whether *agents* comply with policies, whereas we use a policy to audit the *application*. In most of these systems the model delivers the verdict. In StateScout it is not meant to decide which elements exist, but it can still influence a verdict through the tags it adds (Sect. 4.2). VLMs

perform near chance on negated queries [1] and show bag-of-words behaviour [47, 24]; these studies use natural images, so we cite them as motivation, and transferring them to UI yes/no questions is an extrapolation we do not test. nl2spec and NL2LTL [4, 13] translate language into temporal logic; our planned English-to-clause compiler is not implemented, and our policies are supplied in structured form. Sridhar et al. [36, 34] report fire-detection work (by title and venue only; we make no claim about their method) from the same institution; it is listed as adjacent work, not as support for our approach.

3 Problem Formulation

3.1 Applications, Captures and States

We view the application under test (AUT), as experienced by a fixed role r, as a labelled transition system $M_r = (Q, A, \delta, q_0)$ with concrete UI configurations Q, actions A and a partial transition function $\delta : Q \times A \rightharpoonup Q$. The auditor never sees Q itself. It sees a *capture* $\kappa(q) = (u, d, t, s)$ made of the URL, the serialised DOM, the accessibility (AX) tree and optionally a screenshot. One role is fixed per run, so auditing several roles means several runs.

Definition 1 (State fingerprint). *Let N normalise a capture by removing volatile content (session and CSRF tokens, nonces, long numeric timestamps, generated hexadecimal identifiers, per-session CDP node identifiers, transient focus) and let H be SHA-256. The fingerprint is $\phi = H \circ N$, and two configurations are the same abstract state iff $\phi(\kappa(q)) = \phi(\kappa(q'))$.*

A normaliser can be too sensitive, so that every page load is a new node, or too coarse, so that a security-relevant change merges into an existing state and is never audited. Sect. 5.4 probes both with targeted edits; it is a probe, not a measurement of either failure rate.

3.2 The Exploration Graph Is Cyclic

Definition 2 (Exploration graph). *The exploration graph is a directed multigraph $G = (V, E)$ with $V \subseteq \operatorname{img} \phi$ and $E \subseteq V \times A \times V$. An edge (v, a, v') records that executing a in v was observed to lead to v'. We call it a* revisit edge *if v' had already been discovered when the edge was added.*

A revisit edge is any non-tree edge of the discovery order, including cross and forward edges; it is not the depth-first notion of a back edge, although the prototype's field is named `is_back_edge`. Crawlers often drop revisits as redundant. We keep every edge, because facts such as "log out leads to login" describe the AUT's navigation structure and a path-sensitive policy can only be checked on a graph that retains them. The graph is therefore cyclic and revisit edges are never pruned. Loop prevention rests instead on a *claim set* $C \subseteq V \times A$ of visited (state, action) pairs.

Definition 3 (Claim-before-execute). *A frontier action a is executed from v only if $(v, a) \notin C$ when it is enqueued. The pair is inserted into C before a is sent to the browser, and the edge (v, a, v') is persisted only after a has succeeded.*

One browser can only act on the page it is on, so a frontier action from another state is reached by *replaying* its path from the seed, which fires the path's actions again. StateScout limits this by action kind: only $\mathsf{ReplaySafe} = \{\texttt{navigate}, \texttt{click}, \texttt{back}\}$ may be re-fired. A candidate whose path contains a `fill`, `select` or `submit` is recorded as skipped with a reason, and its subtree is reported as unexplored. The residual assumption is that clicks are navigational: a click that mutates state on a replayed path, such as a one-click "Delete", *is* re-fired, which we record as an open limitation. Actions are content-addressed, $id(a) = H(role \,\|\, name \,\|\, selector)$, so one "Log out" control has one identity on every page, while deduplication is scoped to the pair (v, a).

Proposition 1 (Bounded frontier executions, uninterrupted runs). *Assume $|V| \leq n$ (enforced by a state cap), at most k actions per state, a depth bound D on explored states, and a run without crashes. Then the loop terminates; it executes at most nk frontier actions; no pair (v, a) is executed as a frontier action twice; and replay re-fires at most nkD actions in total.*

Proof. Each frontier execution first adds a new pair to C, and $|C| \leq nk$. A pair is enqueued only when its state is first visited and the pair is not yet claimed or queued, so it enters the frontier at most once. Every loop iteration removes one frontier entry, so the loop halts. A replay re-fires the path of one frontier entry, whose length equals the depth of its source state ($\leq D$), and there are at most nk entries.

The proposition is deliberately narrower than "at-most-once": it bounds *frontier* executions of a pair, not the times an action reaches the AUT, since replay re-fires path actions (Sect. 5.2 measures this), and it excludes crashes. The intended behaviour is that a pair claimed but lacking an edge counts as done and is not fired again, at the price of leaving it unexplored; Sect. 5.2 reports a simulated test in which two resume mechanisms disagree about this.

3.3 Policies as Two Set Operations

A QA engineer writes the policy in English, to be compiled into clauses $c = (\sigma_c, \rho_c, \pi_c)$: a subject σ_c, a role scope ρ_c (empty meaning all roles) and a polarity $\pi_c \in \{\mathsf{must_exist}, \mathsf{must_not_exist}\}$. The clauses split into a forbidden set $\mathcal{F}$ and a required set $\mathcal{R}$. Perception maps each state v to a set $\Sigma(v)$ of grounded subjects: element tags, accessible names and capabilities. For role r, $\mathcal{F}_r$ and $\mathcal{R}_r$ denote the clauses whose scope admits r.

Definition 4 (Negation-aware audit).

$$Viol_{\mathcal{F}}(v) = \{\, c \in \mathcal{F}_r : \sigma_c \in \Sigma(v) \,\} \qquad \textit{(forbidden present)} \tag{1}$$

$$Viol_{\mathcal{R}}(v) = \{\, c \in \mathcal{R}_r : \sigma_c \notin \Sigma(v) \,\} \qquad \textit{(required absent)} \tag{2}$$

The audit of v is $Viol_{\mathcal{F}}(v) \cup Viol_{\mathcal{R}}(v)$, with each violation tagged by the operation that produced it.

Equation (1) is an intersection and Eq. (2) a set difference. An oracle built from intersection alone cannot flag a missing element, since a missing element has nothing to intersect with.

3.4 Error Direction Under Imperfect Perception

In practice $\Sigma(v)$ is replaced by an estimate $\hat{\Sigma}(v)$ produced by a DOM/AX correlator or a VLM. The two operations respond to its errors in opposite directions, and this follows from their definitions.

Proposition 2 (Monotonicity of the two operations). *Let $\widehat{Viol}$ be the audit computed from $\hat{\Sigma}$.*

1. *If perception under-reports, $\hat{\Sigma}(v) \subseteq \Sigma(v)$, then $\widehat{Viol}_{\mathcal{F}}(v) \subseteq Viol_{\mathcal{F}}(v)$ and $\widehat{Viol}_{\mathcal{R}}(v) \supseteq Viol_{\mathcal{R}}(v)$: forbidden clauses can only be missed and required clauses can only raise false alarms.*
2. *If perception over-reports, $\hat{\Sigma}(v) \supseteq \Sigma(v)$, both inclusions are reversed.*

Proof. Equation (1) is monotone in Σ and Eq. (2) is antitone in Σ.

This is an immediate property of the definitions, not a research result. Its use is a testable prediction about an implementation: for a perceiver that only drops elements, the engine must produce no forbidden false positives and no required false negatives, and for one that only adds spurious elements, no forbidden false negatives and no required false positives. A violation would mean the engine deviates from Eqs. (1) and (2), for example through substring matching; Sect. 5.3 tests this on the production engine. A perceiver with a single global bias cannot be safe for both clause types, although one with polarity-specific bias could be.

For real channels the direction is relative to the policy vocabulary: a channel that tags an element with a string outside the policy's subject vocabulary has, for that clause, under-reported. That reading makes the direction checkable on a rule-based channel but also means one application cannot refute the observation; the perturbation test is the falsifiable part. Which polarity a VLM errs towards on UI screenshots [1, 47, 24] is an empirical question we leave to the probe set of Sect. 6.

4 The StateScout Prototype

The prototype is a checkpointed LangGraph state machine that reaches the browser, perception and storage only through typed ports, each replaceable by an in-memory fake (the experiments use the fakes). We describe the parts the evaluation relies on.

4.1 Orchestration, Capture and Persistence

Exploration is a breadth-first search over (state, action) pairs. A plain Python reference loop, kept as a parity oracle, has been ported to a LangGraph `StateGraph` (`scan`, `reason`, `act`, `observe`). The checkpointed dictionary holds the frontier, the discovery order, the skips and counters, but not the claim set, which lives in the graph store; the two are separate stores, so resuming a checkpoint does not consult the claims again (Sect. 5.2). Limits such as depth and state cap are configuration, so a resumed run cannot silently change its rules. A skipped action returns straight to `act`, since routing it through `scan` would add a spurious entry to the discovery order, which is the evidence for cycles.

A Playwright crawler records the DOM, the CDP accessibility tree and a screenshot; candidate actions come from actionable AX roles and receive content-addressed identifiers. The normaliser N strips known volatile attributes (`nonce`, `data-session`, `data-csrf` and similar), hexadecimal-looking generated ids, prefixed tokens, runs of ten or more digits, whitespace and volatile CDP keys. These patterns are broad on purpose, and Sect. 5.4 shows what they merge. The claim set lives in Redis and the graph, revisit edges included, in Neo4j. Neither store, nor the browser, is used in the experiments of this paper.

4.2 Perception

Perception turns a capture into a *Semantic UI Map*: elements with role, accessible name, tags, selector, visibility and enabled state, plus the capabilities the state exposes (for example `delete-user`). The deterministic channel correlates DOM elements with AX nodes by browser identifier, then by role plus name, then by unique name, and never guesses at ambiguous matches. DOM-only elements are kept, since an element hidden from the AX tree can still be an exposure.

A VLM channel (a Gemini Flash model behind a rate-limited provider interface) reads the screenshot and AX tree to supply judgements the DOM cannot, for example that a styled `div` looks like a delete button. By design it only annotates elements the deterministic channel already found, adding tags and capabilities; it can neither add nor remove an element. That limits its influence but does not make the verdict immune to it: Eqs. (1) and (2) test membership of a subject in $\Sigma(v)$. Subjects that are identifiers or accessible names come from the DOM and AX tree, but subjects that are tags or capabilities can be created or hidden by a wrong annotation, in either direction, which falls under Proposition 2. The channel is present in the code base but not wired into the orchestrator and *not evaluated* here, so the division of labour is a design hypothesis. Canvas-rendered UI is invisible to the deterministic channel.

4.3 Negation Engine

The engine implements Eqs. (1) and (2) deterministically. It checks that a clause applies to the run's role, then matches subjects against capabilities (the stronger

evidence) and then against element tags and accessible names, exactly after whitespace and case normalisation; substring matching is avoided because an over-eager matcher is itself an over-reporting bias. Each violation carries its clause type, a severity fixed by clause type, a rationale and evidence for a human reviewer. Each test application ships its pages, an `app.json`, a structured `policy.json` beside the English policy, and a `violations.json` of planted defects.

5 Evaluation

5.1 Questions, Subject and Arms

We ask five questions, all at fixture level.

RQ1 (a) Do the production audit and the ground truth agree under oracle perception? (b) What does the exploration loop do: how many pairs does it execute, what does replay cost, are revisit edges kept, and what happens when a side-effecting step or a crash is injected?

RQ2 How does the match between policy vocabulary and perception output affect accuracy, and do the errors of a real channel and of perturbed perceivers follow Proposition 2?

RQ3 Is the required-absent operation necessary? (Answered in the Table 5 row "$\mathcal{R}$ dropped": an absence cannot intersect anything.)

RQ4 Does ϕ keep volatile noise out, and which targeted edits does it fail to separate?

RQ5 On programmatic variants of the fixture, per mutant family, how do the arms behave, and what do two hand-written invariant scripts, one reading the tag encoding and one independent of it, achieve?

Subject and arms. `broken-admin` (Sect. 2.1) is a static six-page application audited as role `guest`; its ground-truth files list four planted violations (V-01 to V-04) and four named cycles. The policy has three forbidden clauses and one required clause. V-03 lies three actions from the seed (index, login, dashboard, admin). A violation is the triple (page, clause, clause type), scored against the planted triples. Table 2 lists the arms. The oracle arms use a fake perceiver that reads the `data-tag` annotations in which the ground truth is itself written, so their agreement concerns clause logic and encoding, not perception. No arm uses a browser, Neo4j or Redis, and the AX tree is used only in the fingerprint tests.

Environment and artifact. The script `experiments/run_experiments.py` writes `data/experiments.json` (both under `docs/report/`), which records Python 3.13.13, LangGraph 1.2.10 and repository commit `4487574` (the code under `apps/`, `tests/` and `test-apps/` was unmodified, but the stored flag `code_tree_clean_vs_head` is false because the E14 fixtures, an untracked folder under `tests/`, exist) and a map from each RQ to its keys. Seed 7 drives all randomised parts. The script, data file and E14 fixtures are not committed and no archived release exists.

Table 2. Experimental arms. *Prod.* is the production negation engine; *fake* is the in-memory fake's own set algebra. RQ4 uses E2 and E8.

Arm	Perception	Audit	RQ: JSON key
A1 oracle, fake algebra	fake, reads `data-tag`	fake	1a, 3: E1_E3
A2 oracle, production	fake, reads `data-tag`	prod.	1a,3: E6; 5: E9
A3 DOM channel	rule-based builder	prod.	2: E1_E3; 5: E9
A4 DOM + lexical	A3, hyphen as space	patched	2: E1_E3
A5 perturbed oracle	A2 maps, perturbed	prod.	2: E7
A6 tag script	HTML parse of `data-tag`	hand-written	5: E9
A7 loop	fake, scripted crawler	fake	1b: E5, E11
A8 XPath script	element types, text	hand-written	5: E10, E12–E14

Table 3. Loop counters (A7); the 25 shuffles give identical counts. "Dup.": pairs executed as a frontier action more than once; "replay/steps": re-navigations and re-fired actions; "revisits": revisit edges. In the last row the login-to-dashboard step is a side-effecting action.

Scenario	States	Pairs	Dup.	Replay/steps	Revisits	Skipped
Uninterrupted; 25 link orders	6	11	0	9/21	6	0
Login step `submit` or `fill`	4	4	0	2/3	1	3

5.2 RQ1: Audit Agreement and Loop Behaviour

(a) Agreement under oracle perception. Both oracle arms report exactly the four planted triples (P, R and F1 of 1.00; Table 5, rows 1–2) and agree page by page. Arm A2 sends the fake's semantic maps through the production `negation.audit.audit`, so the production operators are exercised. It remains a consistency check: both arms read the ground truth's tag encoding, so a perfect score is close to guaranteed, and it is neither a perception measurement nor an end-to-end result.

(b) The exploration loop. We ran the LangGraph loop over a scripted crawler built from `app.json` with the fake perceiver and graph (A7); the framework-free reference loop gave identical counters and graph. Table 3 gives the counters.

The uninterrupted run ends by frontier exhaustion, reaches all 6 states, persists all 11 transitions (5 tree edges, 6 revisit edges), finds all four planted violations and executes no pair twice as a frontier action; violation sets are also identical across link orders. Replay is nonetheless the dominant cost: 21 re-fired actions for 11 frontier executions (1.91 per pair; the bound of Proposition 1 is $11 \cdot 3 = 33$, with $D = 3$ the observed maximum depth and a configured limit of 10), and *index* $\rightarrow$ *login* is executed 10 times, once as a frontier action and nine times as a replay prefix. With `click` counted as replay-safe, that is harmless only if clicks are navigational.

Table 4. Simulated crash and resume at 11 positions. "Twice": positions at which a claimed pair was attempted again after resume. "Graph equal": positions at which the final graph equals the uninterrupted run's (a shortfall means exploration was lost).

Resume mechanism	Crash point	Twice	Graph equal
Checkpoint, same browser	action, after claim	11/11	11/11
Checkpoint, fresh browser	action, after claim	11/11	9/11
Checkpoint, same browser	edge write	0/11	11/11
Checkpoint, fresh browser	edge write	0/11	10/11
Claim set only, new run	action, after claim	0/11	0/11

Making the login step a `submit` or `fill` shows the other side of the rule: every candidate behind it has an unreplayable path and is skipped with a recorded reason, so 4 states are reached, 3 actions are skipped and one of four planted violations (V-01) is found. The rule trades coverage for safety.

Crash and resume. Proposition 1 excludes crashes. We injected an exception in the `act` node right after a claim, and separately in the edge write, at each of the 11 claims (13 on the variant below), using LangGraph's in-memory checkpointer, and resumed in three ways: from the checkpoint with the same fake browser, from it with a fresh crawler that has opened no page, and with no checkpoint, only the persisted claim set, as a new run on the same graph (Table 4). The fake action fails before it has any effect, so we count *attempts*; whether an attempt had an effect in a real AUT is unknowable here.

LangGraph documents that a resumed node restarts from its beginning and that side effects should be idempotent or wrapped in tasks [16], so the mechanism is the framework's documented behaviour. The finding is the gap between it and the claim handling intended in Sect. 3: resuming the checkpoint re-runs the interrupted `act` node, which does not consult the claim set again, so the in-flight pair is attempted twice at all 11 positions; a crash after the action but before the edge is written is handled correctly. A browser that is not where the checkpoint says it is leaves the graph incomplete in 3 of 22 cases. Restarting from the claim set alone never repeats a claimed pair but never continues either: every action of the seed is already claimed, so the new run claims no pair, visits one state, and leaves 0 to 10 of the 11 transitions persisted. That last result is specific to this fixture, whose seed has one action. On a variant with a second seed action (an `About` page; 7 states, 13 pairs; E11) the checkpoint results are the same (13/13 attempted twice after a claim, 0/13 after an edge write), but a claim-set restart now continues through the seed action that was not yet claimed: with a crash at the first claim the new run executes 2 pairs, and from the second claim on it executes none, leaving 1 to 12 of 13 transitions persisted. The positions share one mechanism and are not samples. Claim-before-execute thus gives at-most-once only in the second mode, and only by giving up completeness.

Table 5. Accuracy on `broken-admin` by arm, with errors split by clause type ($\mathcal{F}$: forbidden-present, $\mathcal{R}$: required-absent). Genuine TP excludes a required-absent hit when the clause also fires on pages that carry the element: it then fires everywhere and the hit is a coincidence.

Arm	P	R	F1	Gen. TP	$FN_{\mathcal{F}}$	$FP_{\mathcal{F}}$	$FN_{\mathcal{R}}$	$FP_{\mathcal{R}}$
A1 oracle, fake algebra	1.00	1.00	1.00	4	0	0	0	0
A2 oracle, production	1.00	1.00	1.00	4	0	0	0	0
A1, $\mathcal{R}$ dropped (RQ3: loses V-04)	1.00	0.75	0.86	3	0	0	1	0
A3 DOM channel	0.29	0.50	0.36	1	2	0	0	5
A4 DOM + lexical	1.00	0.50	0.67	2	2	0	0	0

5.3 RQ2: Vocabulary Match, Perception Error and Error Direction

A vocabulary-mismatch case study. Arm A3 builds the semantic map from the HTML with rule-based tagging (Table 5). Its precision of 0.29 and recall of 0.50 do not measure perception quality; they follow from an inconsistency inside the prototype. The policy subjects are `admin-link`, `debug-panel` and `skip-to-content`, but the tagger emits shorter tags for the admin link and the debug section and none equal to the skip subject. V-01 and V-02 are missed, and the skip clause fires on all six pages: five false positives and one *coincidental* hit on V-04. Only V-03 is detected for the right reason, via the delete-user capability, so genuine recall is 0.25, not 0.50.

Arm A4 normalises hyphens inside the matcher (in-process patch). It was designed after seeing A3's errors, on the same four items, so it is an exploratory diagnosis, not a test. It removes the five false positives, so V-04 is now detected for the right reason; V-01 and V-02 stay missed, because deciding that a link labelled "Admin" is an `admin-link` is a semantic judgement. Whether to rename the policy subjects or let perception read the annotations is an unresolved design decision, on which the end-to-end result depends.

A falsifiable test of the error direction. A3 and A4 err in the direction that Proposition 2 reads off for under-reporting, but since a vocabulary mismatch counts as under-reporting by definition, they cannot refute it. We therefore perturbed the oracle maps (A5): under-reporting drops each policy-relevant element with probability p, over-reporting adds each absent policy subject as a spurious element with probability p, and the production audit runs on every perturbed state (200 trials for $p = 0.25$ and 0.5, one deterministic run for $p = 1$; E7). For example, at $p = 0.5$ under-reporting gives on average 1.57 forbidden misses and 2.58 required false alarms, over-reporting 7.79 forbidden false alarms and 0.47 required misses. In all 802 runs, under-reporting produced no forbidden false positive and no required false negative, and over-reporting produced no forbidden false negative and no required false positive. This confirms that the engine is monotone and antitone as Eqs. (1) and (2) require. It says nothing about how a real VLM errs.

5.4 RQ4: Fingerprint Noise and Targeted Edits

Each page is re-rendered 50 times (seed 7) with fresh session and CSRF attributes, a generated id, a request id, a 13-digit timestamp and a CDP-shaped AX tree with new node ids. A naive hash of URL, DOM and AX JSON gives 300 distinct ids for six logical pages, ϕ gives 6. This is a regression check against noise N was written to remove.

The informative part is twelve targeted edits to the `admin` page. Eight change the id (removing or adding a button, renaming a label, swapping links, changing the URL query or an AX name, and, debatably, adding a `hidden` element or editing a comment). Four do not: a whitespace change (by design) and three content changes that fall inside the normaliser's patterns, namely a visible 10-digit number, a visible word with a `req_` prefix, and an identifier-like `id` value (`user-1234`). Those three are over-merges: states that differ only in such content are audited once, which matters when a policy subject depends on that content (not the case here), and the patterns are broad. The hidden-element and comment edits show the opposite risk of over-splitting on pages with hidden or generated markup. We did not measure how often either occurs on real pages.

5.5 RQ5: Mutants, Baselines and Further Apps

We generated variants of `broken-admin` in memory (nothing is written to the test applications): all 24 applicable single operations (remove or add forbidden elements, remove or restore the skip link), 40 seeded compound mutants, and label renames that keep the defects but change visible text. A generator ledger gives each variant's ground truth independently of the audit code. Two hand-written scripts, our own re-implementations and neither Crawljax nor ATUSA, serve as baselines. A6 reads `data-tag`, like the oracle arms. A8 does not: it has four invariants in the XPath subset of Python's ElementTree over element types, link targets and visible text (for instance, an `a` element whose text is "Admin"), plus a 26-line shared HTML-to-tree helper. The authors knew the labels when writing it; it was written before its first run on stripped pages, which we state but cannot evidence, as the artifact has no history. Every variant is also run on a copy with the `data-tag` attributes removed.

On the 64 shape-preserving variants A2, A6 and A8 report exactly the ledger and A3 matches it on none. A2 and A6 agree because both read the tag spelling in which the mutants are written, so 64/64 shows consistency, not advantage. On unannotated pages the informative comparison is A3 against A8: a short label-based script beats the shipped DOM channel (stripped original fixture: A3 2 true and 5 false positives, A8 4 and 0), because A3's vocabulary does not match the policy subjects (Sect. 5.3). A8's exactness is not independent evidence, since the mutator inserts the label texts A8 was written against. When visible text changes it loses: of three independent renames, "Admin" and "Delete user" each cost A8 one missed forbidden violation and "Skip navigation" makes the skip clause fire on five pages, while A2 and A6 are unaffected; tag-spelling renames (E9) are the mirror image. Each encoding fails where the other does not. The

Table 6. Errors per mutant family, as $FN_{\mathcal{F}}/FP_{\mathcal{R}}$ summed over the family's n variants (no arm produced $FP_{\mathcal{F}}$ or $FN_{\mathcal{R}}$ anywhere). "Planted": forbidden/required violations in the family. A2 and A6 on intact tags make no error in any family, and A8 on stripped copies equals A8 on intact ones. A2s is the oracle perceiver run without its annotations (stripped copies), shown as a reference and not as a competitor. The label-rename row includes the combined variant, so its A8 totals double-count the singles.

Family	n	Planted	A3	A2s	A8
Remove planted forbidden	3	6/3	4/15	6/15	0/0
Add forbidden element	15	60/15	40/75	60/75	0/0
Remove or restore skip link	6	18/10	12/26	18/26	0/0
Compound	40	162/47	113/193	162/193	0/0
Label rename	4	12/4	8/20	12/20	4/10

68 variants rearrange the same six pages, so their counts are not independent samples.

Three further apps. The repository holds three more apps from the same project, `broken-rbac-demo`, `missing-consent-demo` and `broken-feature-flag-demo`: one to four static, unannotated pages and one documented defect each; we read them and modified nothing. Clauses and ground-truth triples were derived from each README sentence first (the ordering is an authorial statement the artifact cannot evidence), and the XPath invariants were written after reading the scored HTML, so A8's results are in-sample and say nothing about generalisation (E12). A8 finds all three defects with no false positive. A3, with README-derived subjects, finds only the consent one, vacuously: the clause fires on any page, since the tagger has no consent vocabulary, and still fires when a banner is inserted in memory; the consent row therefore does not discriminate for A8 either. A3 misses the other two for the vocabulary reason of RQ2; with subjects respelled in the tagger's vocabulary (post hoc) it finds both but also flags `admin.html`. The truth is narrower than the README text: the feature-flag policy sentence also names an analytics panel, which A8's exact-heading query misses (under a truth that includes it, A8 finds 1 of 2 and A3 0 of 2), and the RBAC dashboard leaks a delete button no clause covered.

Four generated apps (E14). Four apps (`shop-hub`, `med-clinic`, `hr-suite`, `config-portal`; 7 unannotated pages each, three violations and clean-page lists each) were generated by an AI tool within the same project. They add variety and a more blind protocol, not independence. The A8 invariants were written from the policy files only, before opening any page, ground truth or README, and saved with their SHA-256 and a UTC timestamp (E14). The order is self-attested: the timestamp is a file time, all fixtures existed before it, and the hash shows only that the file was not edited afterwards. A3 uses the policy subjects verbatim, and nothing was changed after the results.

The fixtures are weak. The ground truth is the generator's own assertion and is unvalidated. Six of the eight forbidden-violation pages have no inbound link, login pages link nowhere, and roles are nominal in static pages. Several clean-page labels contradict the policy wording or the HTML: the shop-hub admin clause speaks of a *link* that exists nowhere; shop-3 holds app-wide through the Support link yet flags only the home page; med-3 lists `appointments.html` as clean although it lacks emergency information; cfg-3 lists `settings.html` as clean although it has an organisation field and a save button; and hr-3 flags the benefits page although the dashboard shows the status. False-positive counts are therefore only as meaningful as these lists.

A3 finds none of the 8 forbidden violations. On required clauses it fires on every page, so its 4 hits are coincidental, with 19 false positives on listed clean pages and 33 hits on unlisted pages (neither truth nor clean, so not counted as false positives), because its vocabulary matches none of the subjects. A8 finds 3 of 8 forbidden violations (checkout, staff-portal and team pages) with no false positive; it misses five, where the violation is a whole page reached without a link in the form it guessed or the label differs from its guess ("Run Payroll Calculation"). Its raw required tally is 3 of 4 with 4 false positives and 12 unlisted hits, but applying Table 5's genuine-TP rule as for A3 removes two of the three hits (the clause also fires on listed pages that carry the element), and the third, on `config-portal`, is vacuous: no fixture page has an `id` attribute, so its `id` queries can never match. A8's genuine required recall is thus 0 of 4 to 1 of 4, depending on how "carry" is read, and it misses the benefits-status violation because it tests for a link, not the status display. A8's invariants also carry page scope ("the home page", "not the login page") that the engine lacks, which A3 cannot express. All of A8's false positives and 7 of A3's fall on the three double-negation clauses ("must not fail to find", "should never be unable to find"), encoded as required presence as the policy file does. A hand-written label script thus helps only where its author guessed the labels, a more modest result than on `broken-admin`.

5.6 Threats to Validity

This is the single statement of the evidence's limits. *External.* The evidence is one small, static, hand-built application with four planted defects and its variants, plus three tiny single-defect apps and four generated apps (untracked fixtures of weak soundness), all from the same project; it shows that the mechanisms behave as designed on these and is not independent validation. No genuinely independent subject and no real Crawljax-plus-ATUSA baseline were available; A8 is a partial, self-written substitute. *Construct.* The oracle arms read the ground truth's own encoding; A3 and A4 work on HTML with no AX tree; the loop experiments use fakes, and the crash test injects in-process exceptions with an in-memory checkpointer and a fake browser, so it probes loop logic and not Redis, Neo4j or Playwright behaviour. No browser-driven, VLM or hybrid result is reported. *Internal.* The authors wrote the application, policy, tool, mutation operators and both baselines, knew the labels A8 queries on `broken-admin`

(not on the E14 apps), and chose the rename variants; A4 was designed after inspecting A3's errors. *Statistical.* Runs are deterministic or use one seed, without confidence intervals; positive counts are small.

6 Discussion and Next Steps

Resuming safely. Since LangGraph re-runs a node on resume [16], the crash test suggests a concrete change: `act` should consult the claim set and treat a claimed pair with no recorded edge as *in doubt*, skipping it with a recorded reason or handing it to a human, not re-firing it silently. A claim-set restart needs the opposite repair, rebuilding the frontier from persisted edges. Both need a durable checkpointer and a real browser to test.

Subject grounding. Accuracy hinges on mapping a policy subject to the elements and capabilities of a rendered state. A VLM could help there, but hallucination and negation weakness would enter through the tags it adds; Proposition 2 suggests scoring it per polarity on a labelled probe set of hard cases (unlabelled `div` controls, hidden and icon-only elements). An *indeterminate* verdict when channels disagree, escalated to a human, is a proposal only.

Scope, replay safety and what is missing. Clauses are scoped by role only; page-scope predicates and path-sensitive properties need the revisit edges we keep. Treating `click` as replay-safe assumes navigational clicks. The results do not show that a browser-driven run would match the oracle. Beyond the fixture, the work needs independent subjects, a real Crawljax-plus-invariants baseline, a browser-driven run once the vocabulary question is settled, the VLM probe set and an archived artifact; none has been run.

7 Conclusion

Treating the policy oracle as two typed set operations makes a missing element a first-class violation. On one six-page fixture the production audit agrees with the ground truth under oracle perception, the loop keeps its revisit edges, and perturbed perceivers err as the set algebra implies. Three results qualify the design: a vocabulary mismatch, fingerprint over-merges, and unsafe resumption. The evidence is fixture-level (Sect. 5.6); the browser-driven pipeline and the VLM channel remain unevaluated, and next steps are in Sect. 6.

Acknowledgments

The reference list additionally includes [35, 33], listed here without further claim.

Bibliography

[1] Kumail Alhamoud, Shaden Alshammari, Yonglong Tian, Guohao Li, Philip H. S. Torr, Yoon Kim, and Marzyeh Ghassemi. Vision-language models do not understand negation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[2] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015.

[3] Jieshan Chen, Jiamou Sun, Sidong Feng, Zhenchang Xing, Qinghua Lu, Xiwei Xu, and Chunyang Chen. Unveiling the tricks: Automated detection of dark patterns in mobile applications. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, pages 1–20, 2023.

[4] Matthias Cosler, Christopher Hahn, Daniel Mendoza, Frederik Schmitt, and Caroline Trippel. nl2spec: Interactively translating unstructured natural language to temporal logics with large language models. In *Computer Aided Verification (CAV)*, pages 383–396. Springer, 2023.

[5] Valentin Dallmeier, Martin Burger, Tobias Orth, and Andreas Zeller. WebMate: Generating test cases for web 2.0. In *Software Quality. Increasing Value in Software and Systems Development (SWQD 2013)*, pages 55–69. Springer, 2013.

[6] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2Web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[7] Deque Systems. axe-core: an accessibility testing engine for websites and other HTML-based user interfaces. Software repository. Accessed 2026-10-09.

[8] Deque Systems. axe rule `bypass`: page must have means to bypass repeated blocks (axe 4.10). Deque University rule documentation. Accessed 2026-10-09.

[9] Deque Systems. axe rule `marquee`: `<marquee>` elements are deprecated and must not be used (axe 4.10). Deque University rule documentation. Accessed 2026-10-09.

[10] I Putu Arya Dharmaadi, Mohannad Alhanahnah, Van-Thuan Pham, Fadi Mohsen, and Fatih Turkmen. BACFuzz: Exposing the silence on broken access control vulnerabilities in web applications, 2025. arXiv:2507.15984 (preprint).

[11] Adam Doupé, Ludovico Cavedon, Christopher Kruegel, and Giovanni Vigna. Enemy of the state: A state-aware black-box web vulnerability scanner. In *Proceedings of the 21st USENIX Security Symposium*, 2012.

[12] Marcelo Medeiros Eler, José Miguel Rojas, Yan Ge, and Gordon Fraser. Automated accessibility testing of mobile apps. In *IEEE 11th International*

Conference on Software Testing, Verification and Validation (ICST), pages 116–126, 2018.

[13] Francesco Fuggitti and Tathagata Chakraborti. NL2LTL – a Python package for converting natural language (NL) instructions to linear temporal logic (LTL) formulas. In *Proceedings of the AAAI Conference on Artificial Intelligence (System Demonstrations)*, volume 37, pages 16428–16430, 2023. Issue 13.

[14] Tianxiao Gu, Chengnian Sun, Xiaoxing Ma, Chun Cao, Chang Xu, Yuan Yao, Qirun Zhang, Jian Lu, and Zhendong Su. Practical GUI testing of Android applications via model abstraction and refinement. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pages 269–280, 2019.

[15] Yongxiang Hu, Xuan Wang, Yingchuan Wang, Yu Zhang, Shiyu Guo, Chaoyi Chen, Xin Wang, and Yangfan Zhou. AUITestAgent: Automatic requirements oriented GUI function testing, 2024. arXiv:2407.09018.

[16] LangGraph documentation: durable execution. Project documentation. Accessed 2026-10-09. The page now redirects to the Persistence page; the statements on node restart and idempotent side effects were read as of 2025-12-13 in the archived copy https://web.archive.org/web/20251213033841/https://docs.langchain.com/oss/python/langgraph/durable-execution and are repeated on the current interrupts (https://docs.langchain.com/oss/python/langgraph/interrupts) and functional-api (https://docs.langchain.com/oss/python/langgraph/functional-api) pages.

[17] Ido Levy, Ben Wiesel, Sami Marreed, Alon Oved, Avi Yaeli, Nir Mashkif, and Segev Shlomov. ST-WebAgentBench: A benchmark for evaluating safety and trustworthiness in web agents. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2410.06703.

[18] Xiaowei Li, Xujie Si, and Yuan Xue. Automated black-box detection of access control vulnerabilities in web applications. In *Proceedings of the 4th ACM Conference on Data and Application Security and Privacy (CODASPY)*, pages 49–60, 2014.

[19] Yuanchun Li, Ziyue Yang, Yao Guo, and Xiangqun Chen. DroidBot: A lightweight UI-guided test input generator for Android. In *Proceedings of the 39th International Conference on Software Engineering Companion (ICSE-C)*, pages 23–26, 2017.

[20] Yuanchun Li, Ziyue Yang, Yao Guo, and Xiangqun Chen. Humanoid: A deep learning-based approach to automated black-box Android app testing. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1070–1073, 2019.

[21] Fengyu Liu, Yuan Zhang, Enhao Li, Wei Meng, Youkun Shi, Qianheng Wang, Chenlin Wang, Zihan Lin, and Min Yang. BACScan: Automatic black-box detection of broken-access-control vulnerabilities in web applications. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1320–1333, 2025.

[22] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Xing Che, Dandan Wang, and Qing Wang. Make LLM a testing expert: Bringing

human-like interaction to mobile GUI testing via functionality-aware decisions. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, pages 1–13, 2024.

[23] Zhe Liu, Cheng Li, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Yawen Wang, Jun Hu, and Qing Wang. Seeing is believing: Vision-driven non-crash functional bug detection for mobile apps, 2024. arXiv:2407.03037, v2 (v1 title: "Vision-driven Automated Mobile GUI Testing via Multimodal Large Language Model", approach VisionDroid).

[24] Zixian Ma, Jerry Hong, Mustafa Omer Gul, Mona Gandhi, Irena Gao, and Ranjay Krishna. CREPE: Can vision-language foundation models reason compositionally? In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10910–10921, 2023.

[25] Ke Mao, Mark Harman, and Yue Jia. Sapienz: Multi-objective automated testing for Android applications. In *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA)*, pages 94–105, 2016.

[26] Atif M. Memon, Ishan Banerjee, and Adithya Nagarajan. GUI ripping: Reverse engineering of graphical user interfaces for testing. In *Proceedings of the 10th Working Conference on Reverse Engineering (WCRE)*, pages 260–269, 2003.

[27] Atif M. Memon, Martha E. Pollack, and Mary Lou Soffa. Automated test oracles for GUIs. In *Proceedings of the 8th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 30–39, 2000.

[28] Ali Mesbah, Arie van Deursen, and Stefan Lenselink. Crawling Ajax-based web applications through dynamic analysis of user interface state changes. *ACM Transactions on the Web*, 6(1):3:1–3:30, 2012.

[29] Ali Mesbah, Arie van Deursen, and Danny Roest. Invariant-based automatic testing of modern web applications. *IEEE Transactions on Software Engineering*, 38(1):35–53, 2012.

[30] Maliheh Monshizadeh, Prasad Naldurg, and V. N. Venkatakrishnan. MACE: Detecting privilege escalation vulnerabilities in web applications. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 690–701, 2014.

[31] George Noseevich and Andrew Petukhov. Detecting insufficient access control in web applications. In *First SysSec Workshop (SysSec)*, 2011.

[32] OWASP Foundation. A01:2021 – broken access control. OWASP Top 10:2021, 2021.

[33] R. R. Sathiya, S. Anton Tamilselvan, M. Neranjan, B. Ashwin, and P. Jayakrishna. Holistic stress and physiological monitoring using video analysis. In *Power Engineering and Intelligent Systems: Proceedings of PEIS 2025, Volume 1*, volume 1459 of *Lecture Notes in Electrical Engineering*, pages 373–386, Singapore, 2026. Springer Nature Singapore.

[34] P. Sridhar, Latha Parameswaran, and Senthil Kumar Thangavel. An efficient rule based algorithm for fire detection on real time videos. *Journal of Computational and Theoretical Nanoscience*, 17(1):308–315, 2020.

[35] P. Sridhar and R. R. Sathiya. Cloud workload forecasting via latency-aware time series clustering-based scheduling technique. *Concurrency and Computation: Practice and Experience*, 37(15–17):e70151, 2025.

[36] P. Sridhar, Senthil Kumar Thangavel, and Latha Parameswaran. A new approach for fire pixel detection in building environment using vision sensor. In *Innovations in Computational Intelligence and Computer Vision: Proceedings of ICICV 2020*, pages 392–400, Singapore, 2020. Springer Singapore.

[37] Ting Su, Guozhu Meng, Yuting Chen, Ke Wu, Weiming Yang, Yao Yao, Geguang Pu, Yang Liu, and Zhendong Su. Guided, stochastic model-based GUI testing of Android apps. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pages 245–256, 2017.

[38] Fangqi Sun, Liang Xu, and Zhendong Su. Static detection of access control vulnerabilities in web applications. In *Proceedings of the 20th USENIX Security Symposium*, 2011.

[39] Maryam Taeb, Amanda Swearngin, Eldon Schoop, Ruijia Cheng, Yue Jiang, and Jeffrey Nichols. AXNav: Replaying accessibility tests from natural language. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI)*, pages 1–16, 2024.

[40] Xiwen Teoh, Yun Lin, Duc-Minh Nguyen, Ruofei Ren, Wenjie Zhang, and Jin Song Dong. WebTestPilot: Agentic end-to-end web testing against natural language specification by inferring oracles with symbolized GUI elements. *Proceedings of the ACM on Software Engineering*, 3(FSE):1933–1956, 2026.

[41] W3C Web Accessibility Initiative. Understanding success criterion 2.4.1: Bypass blocks, WCAG 2.2. W3C documentation. Accessed 2026-10-09.

[42] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. AutoDroid: LLM-powered task automation in Android. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom)*, pages 543–557, 2024.

[43] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[44] Yiheng Xiong, Ting Su, Jue Wang, Jingling Sun, Geguang Pu, and Zhendong Su. General and practical property-based testing for Android apps. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 53–64, 2024.

[45] Rahulkrishna Yandrapally, Andrea Stocco, and Ali Mesbah. Near-duplicate detection in web app model inference. In *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*, pages 186–197, 2020.

[46] Juyeon Yoon, Robert Feldt, and Shin Yoo. Intent-driven mobile GUI testing with autonomous large language model agents. In *Proceedings of the IEEE Conference on Software Testing, Verification and Validation (ICST)*, pages 129–139, 2024.

[47] Mert Yuksekgonul, Federico Bianchi, Pratyusha Kalluri, Dan Jurafsky, and James Zou. When and why vision-language models behave like bags-of-words, and what to do about it? In *International Conference on Learning Representations (ICLR)*, 2023.

[48] ZAP documentation: passive scan rules (including Information Disclosure: Debug Errors, rule 10023). Project documentation. Accessed 2026-10-09.

[49] ZAP documentation: the spider. Project documentation. Accessed 2026-10-09.

[50] ZAP documentation: users within a context. Project documentation. Accessed 2026-10-09.

[51] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. GPT-4V(ision) is a generalist web agent, if grounded. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, 2024.

[52] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024.